

Chapter 2. Python Environments

At its core, every Python installation consists of two things: an interpreter and modules. The modules, in turn, come from the standard library and from third-party packages, if you've installed any. Together, these provide the essential components you need to execute a Python program: a *Python environment* ([Figure 2-1](#)).

Python installations aren't the only kind of Python environment. *Virtual environments* are stripped-down environments that share the interpreter and the standard library with a full installation. You use them to install third-party packages for a specific project or application while keeping the system-wide environment pristine.

NOTE

This book uses *Python environment* as an umbrella term that includes both system-wide installations and virtual environments. Beware that some people only use the term for project-specific environments, like virtual environments or Conda environments.

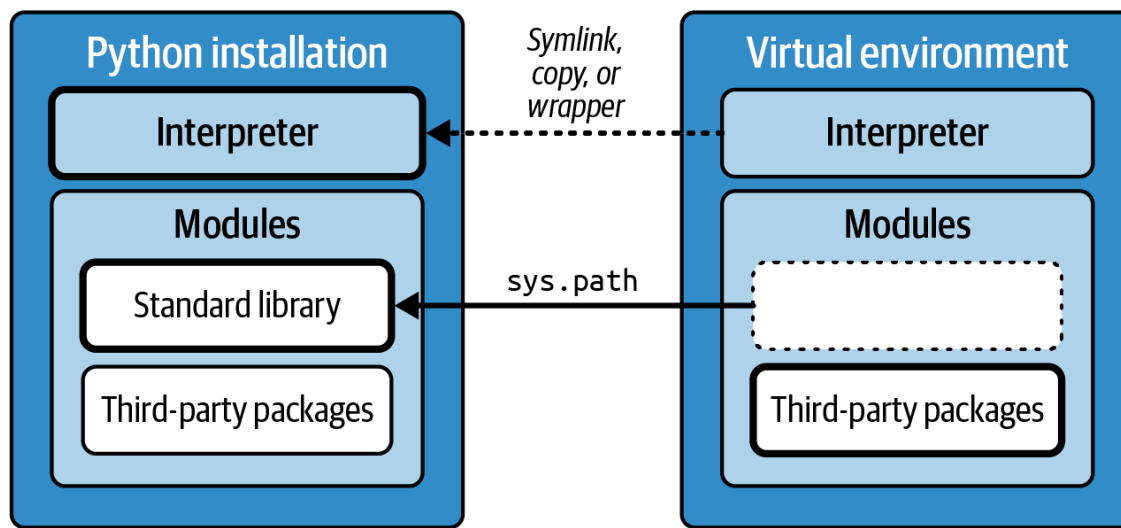


Figure 2-1. Python environments consist of an interpreter and modules. Virtual environments share the interpreter and the standard library with their parent environment.

Managing environments is a crucial aspect of Python development. You'll want to make sure your code works on your users' systems, particularly across the language versions you support, and possibly across major versions of an important third-party package. A Python environment can contain only a single version of each third-party package—if two projects require different versions of the same package, they can't be installed side by side. That's why it's considered good practice to install every Python application, and every project you work on, in a dedicated virtual environment.

In this chapter, you'll build a deeper understanding of what Python environments are and how they work. The chapter has three parts:

- The first part introduces the three kinds of Python environments—Python installations, the per-user environment, and virtual environments—as well as two fundamental tools: the Python package installer `pip` and the standard `venv` module.
- The second part introduces two modern tools that let you manage environments more efficiently: `pipx`, an installer for Python applications, and `uv`, a drop-in replacement for Python packaging tools written in Rust.
- The final part of this chapter dives deep into how and where Python finds the modules you import—feel free to skip it unless you're curious how this works.

NOTE

This chapter uses the Python Launcher to invoke the interpreter (see [“The Python Launcher for Windows”](#) and [“The Python Launcher for Unix”](#)). If you don’t have it installed, replace `py` with `python3` when running the examples.

A Tour of Python Environments

Every Python program runs “inside” a Python environment: the interpreter in the environment executes the program’s code, and `import` statements load modules from the environment. You select the environment by launching its interpreter.

Python offers two mechanisms for running a program on an interpreter. You can pass a Python script as an argument:

```
$ py hello.py
```

Alternatively, you can pass a module with the `-m` option, provided that the interpreter can import the module:

```
$ py -m hello
```

Most commonly, the interpreter imports *hello.py* from the environment—but for the sake of this example, placing it in the current directory will also work.

Additionally, many Python applications install an entry-point script in your `PATH` (see [“Entry-point scripts”](#)). This mechanism allows you to launch the application without specifying an interpreter. Entry-point scripts always use the interpreter from the environment in which they’re installed:

```
$ hello
```

This method is convenient, but there’s also a drawback: if you’ve installed the program in multiple environments, the first environment on `PATH` “wins.” In such a scenario, the form `py -m hello` offers you more control.

“The interpreter determines the environment.” As mentioned previously, this rule applies when you import modules. It also applies in the complementary situation: when you install a package into an environment. Pip, the Python package installer, installs packages into its own environment by default. In other words, you select the target environment for a package by running pip on the interpreter in that environment.

For this reason, the canonical way to install a package with pip uses the `-m` form:

```
$ py -m pip install <package>
```

Alternatively, you can provide a virtual environment or interpreter to pip using its `--python` option:

```
$ pip --python=<env> install <package>
```

The second method has the advantage of not requiring pip in every environment.

Python Installations

This section takes you on a tour of a Python installation. Feel free to follow along on your own system. [Table 2-1](#) shows the most common locations—replace 3.x and 3x with the Python feature version, such as 3.12 and 312.

Table 2-1. Locations of Python installations

Platform	Python installation
Windows (single-user)	<i>%LocalAppData%\Programs\Python\Python3x</i>
Windows (multi-user)	<i>%ProgramFiles%\Python3x</i>
macOS (Homebrew)	<i>/opt/homebrew/Frameworks/Python.framework/Versions/3.x^a</i>
macOS (python.org)	<i>/Library/Frameworks/Python.framework/Versions/3.x</i>
Linux (generic)	<i>/usr/local</i>
Linux (pack- age manager)	<i>/usr</i>

^a Homebrew on macOS Intel uses */usr/local* instead of */opt/homebrew*.

An installation might be cleanly separated from the rest of your system, but not necessarily. On Linux, it goes into a shared location like */usr* or */usr/local*, with its files scattered across the filesystem. Windows systems, on the other hand, keep all files in a single place. Framework builds on macOS are similarly self-contained, although distributions may also install symbolic links into the traditional Unix locations.

In the following sections, you'll take a closer look at the core parts of Python installations—the interpreter and the modules, as well as some other components such as entry-point scripts and shared libraries.

The layout of Python installations varies quite a bit from system to system. The good news is, you rarely have to care—a Python interpreter knows its environment. For reference, [Table 2-2](#) provides a baseline for installation layouts on the major platforms. All paths are relative to the installation root.

Table 2-2. Layout of Python installations

Files	Windows	Linux and macOS	Notes
Interpreter	<i>python.exe</i>	<i>bin/python3.x</i>	
Standard library	<i>Lib</i> and <i>DLLs</i>	<i>lib/python3.x</i>	Extension modules are located under <i>DLLs</i> on Windows. Fedora places the standard library under <i>lib64</i> instead of <i>lib</i> .
Third-party packages	<i>Lib\site-packages</i>	<i>lib/python3.x/site-packages</i>	Debian and Ubuntu put packages in <i>dist-packages</i> . Fedora places extension modules under <i>lib64</i> instead of <i>lib</i> .
Entry-point scripts	<i>Scripts</i>	<i>bin</i>	

The interpreter

The executable that runs Python programs is named *python.exe* on Windows and located at the root of a full installation.¹ On Linux and macOS, the interpreter is named *python3.x* and stored in the *bin* directory with a *python3* symbolic link.

The Python interpreter ties the environment to three things:

- A specific version of the Python language
- A specific implementation of Python
- A specific build of the interpreter

The implementation might be [CPython](#), the reference implementation of Python, but it could also be any of a number of alternative implementations—such as [PyPy](#), a fast interpreter with just-in-time compilation, written in Python itself; or [GraalPy](#), a highly performant implementation with Java interoperability, using the GraalVM development kit.

Builds differ in their CPU architecture—for example, 32-bit versus 64-bit, or Intel versus Apple silicon—and their build configuration, which determines things like compile-time optimizations or the installation layout.

QUERYING THE INTERPRETER ABOUT THE PYTHON ENVIRONMENT

In an interactive session, import the `sys` module and inspect the following variables:

`sys.version_info`

The version of the Python language, represented as a *named tuple* with the major, minor, and micro versions, as well as the release level and serial number for prereleases

`sys.implementation.name`

The implementation of Python, such as `"cpython"` or `"pypy"`

`sys.implementation.version`

The version of the implementation, same as `sys.version_info` for CPython

`sys.executable`

The location of the Python interpreter

`sys.prefix`

The location of the Python environment

`sys.path`

The list of directories searched when importing Python modules

The command `py -m sysconfig` prints a great deal of metadata compiled into the Python interpreter, such as the instruction set architecture, the build configuration, and the installation layout.

Python modules

Modules are containers of Python objects that you load via the `import` statement. They're organized below *Lib* (Windows) or *lib/python3.x* (Linux and macOS) with some platform-dependent variations. Third-party packages go into a subdirectory named *site-packages*.

Modules come in various forms and shapes. If you've worked with Python, you've likely used most of them already. Let's go over the different kinds:

Simple modules

In the simplest case, a *module* is a single file containing Python source code. The statement `import string` executes the code in *string.py* and binds the result to the name `string` in the local scope.

Packages

Directories with `__init__.py` files are known as *packages*—they allow you to organize modules in a hierarchy. The statement `import email.message` loads the `message` module from the `email` package.

Namespace packages

Directories with modules but no `__init__.py` are known as *namespace packages*. You use them to organize modules in a common namespace such as a company name (say `acme.unicycle` and `acme.roocketsled`). Unlike with regular packages, you can distribute each module in a namespace package separately.

Extension modules

Extension modules, such as the `math` module, contain native code compiled from low-level languages like C. They're shared libraries² with a special entry point that lets you import them as modules from Python. People write them for performance reasons or to

make existing C libraries available as Python modules. Their names end in `.pyd` on Windows, `.dylib` on macOS, and `.so` on Linux.

Built-in modules

Some modules from the standard library, such as the `sys` and `builtins` modules, are compiled into the interpreter. The variable `sys.builtin_module_names` lists all of these modules.

Frozen modules

Some modules from the standard library are written in Python but have their bytecode embedded in the interpreter. Originally, only core parts of `importlib` got this treatment. Recent versions of Python freeze every module that's imported during interpreter startup, such as `os` and `io`.

NOTE

The term *package* carries some ambiguity in the Python world. It refers both to modules and to the artifacts used for distributing modules, also known as *distributions*. Unless stated otherwise, this book uses *package* as a synonym for *distribution*.

Bytecode is an intermediate representation of Python code that is platform-independent and optimized for fast execution. The interpreter compiles pure Python modules to bytecode when it loads them for the first time. Bytecode modules are cached in the environment in `.pyc` files under `__pycache__` directories.

You can find out where a module comes from using `importlib` from the standard library. Every module has an associated `ModuleSpec` object whose `origin` attribute contains the location of the source file or dynamic library for the module, or a fixed string like `"built-in"` or `"frozen"`. The `cached` attribute stores the location of the bytecode for a pure Python module. [Example 2-1](#) shows the origin of each module in the standard library.

Example 2-1. Listing standard library modules and their origin

```
import importlib.util
import sys

for name in sorted(sys.stdlib_module_names):
    if spec := importlib.util.find_spec(name):
        print(f"{name:30} {spec.origin}")
```

Environments also store metadata about installed third-party packages, such as their authors, licenses, and versions. [Example 2-2](#) shows the version of each package in the environment using `importlib.metadata` from the standard library.

Example 2-2. Listing packages installed in the environment

```
import importlib.metadata

distributions = importlib.metadata.distributions()
for distribution in sorted(distributions, key=lambda d: d.name):
    print(f"{distribution.name:30} {distribution.version}")
```

Entry-point scripts

An *entry-point script* is an executable file in *Scripts* (Windows) or *bin* (Linux and macOS) with a single purpose: it launches a Python application by importing the module with its entry-point function and calling that function.

This mechanism has two key benefits. First, you can launch the application in a shell by running a simple command—say, `pydoc3` for Python’s built-in documentation browser.³ Second, entry-point scripts use the interpreter and modules from their environment, sparing you surprises about wrong Python versions or missing third-party packages.

Package installers, like `pip`, can generate entry-point scripts for third-party packages they install. Package authors only need to designate the function that the script should invoke. This is a handy method to provide an executable for a Python application (see [“Entry-Point Scripts”](#)).

Platforms differ in how they let you execute entry-point scripts directly. On Linux and macOS, they’re regular Python files with *execute* permission, such as the one shown in [Example 2-3](#). Windows embeds the Python code in a binary file in the Portable Executable (PE) format—more commonly known as a `.exe` file. The binary launches the interpreter with the embedded code.

Example 2-3. The entry-point script `pydoc3` from a Linux installation

```
#!/usr/local/bin/python3.12 ❶
import pydoc ❷
if __name__ == "__main__": ❸
    pydoc.cli() ❹
```

- ❶ Request the interpreter from the current environment.
- ❷ Load the module containing the designated entry-point function.
- ❸ Check that the script wasn’t imported from another module.
- ❹ Finally, call the entry-point function to start up the program.

NOTE

The `#!` line is known as a *shebang* on Unix-like operating systems. When you run the script, the program loader uses the line to locate and launch the interpreter. The *program loader* is the part of the operating system that loads a program into main memory.

Other components

Besides the interpreter, modules, and scripts, Python environments can include some additional components:

Shared libraries

Python environments sometimes contain shared libraries that aren't extension modules, named *.dll* on Windows, *.dylib* on macOS, and *.so* on Linux. Third-party packages may bundle shared libraries they use so you don't need to install them separately. Python installations may also bundle shared libraries—for example, the standard `ssl` module uses OpenSSL, an open source library for secure communication.

Header files

Python installations contain header files for the *Python/C API*, an application programming interface for writing extension modules or embedding Python as a component in a larger application. They're located under *Include* (Windows) or *include/python3.x* (Linux and macOS).

Static data

Python installations also contain static data in various locations. This includes configuration files, documentation, and any resource files shipped with third-party packages.

Tcl/Tk

By default, Python installations also include *Tcl/Tk*, a toolkit for creating graphical user interfaces (GUIs) written in Tcl. The standard

`tkinter` module allows you to use this toolkit from Python.

The Per-User Environment

The *per-user environment* allows you to install third-party packages for a single user. It offers two main benefits over installing packages system-wide: you don't need administrative privileges to install packages, and you don't affect other users on a multiuser system.

The per-user environment is located in the home directory on Linux and macOS and in the app data directory on Windows (see [Table 2-3](#)). It contains a *site-packages* directory for every Python version. Entry-point scripts are shared across Python versions, except on macOS, where the entire per-user installation is kept under a version-specific directory.^{[4](#)}

Table 2-3. Location of per-user directories

Platform	Third-party packages	Entry-point scripts
Windows	%AppData%\Python\Python3x\site-packages	%AppData%\Python\Scripts
macOS	~/Library/Python/3.x/lib/python/site-packages	~/Library/Python/3.x/bin
Linux	~/local/lib/python3.x/site-packages ^{a}	~/local/bin

^{[a](#)} Fedora places extension modules under *lib64*.

You install a package into the per-user environment using `py -m pip install --user <package>`. If you invoke `pip` outside a virtual environment and `pip` finds that it can't write to the system-wide installation, it will also default to this location. If the per-user environment doesn't exist yet, `pip` creates it for you.

TIP

The per-user script directory may not be on `PATH` by default. If you install applications into the per-user environment, remember to edit your shell profile to update the search path. Pip issues a friendly reminder when it detects this situation.

Per-user environments have an important shortcoming: by design, they're not isolated from the global environment. You can still import system-wide site packages if they're not shadowed by per-user modules with the same name. Applications in the per-user environment also aren't isolated from each other—in particular, they can't depend on incompatible versions of another package. Even applications in the system-wide environment can import modules from the per-user environment.

And there's another drawback: you can't install packages into the per-user environment if the Python installation is marked as *externally managed*—for example, if you installed Python using the package manager of your distribution.

In [“Installing Applications with pipx”](#), I'll introduce pipx, which lets you install applications in isolated environments. It uses the per-user script directory to put applications onto your search path but relies on virtual environments under the hood.

Virtual Environments

When you're working on a Python project that uses third-party packages, it's usually a bad idea to install these packages into the system-wide or per-user environments. First, you're polluting a global namespace. Testing and debugging your projects gets a lot easier when you run them in isolated and reproducible environments. Second, if two projects depend on conflicting versions of the same package, a single environment isn't even an option. Third, as mentioned in the previous section, you can't install packages into environments marked as *externally managed*. This is a good thing: installing and uninstalling Python packages behind your package manager's back introduces a real chance of breaking your system.

Virtual environments were invented to solve these problems. They’re isolated from the system-wide installation and from each other. Under the hood, a virtual environment is a lightweight Python environment that stores third-party packages and delegates most other things to a full installation. Packages in virtual environments are visible only to the interpreter in the environment.

You create a virtual environment with the command `py -m venv <directory>`. The last argument is the location where you want the environment to exist—its root directory—and conventionally named `.venv`.

The directory tree of a virtual environment looks much like a Python installation, except that some files are missing, most notably the entire standard library. [Table 2-4](#) shows the standard locations within a virtual environment.

Table 2-4. Layout of a virtual environment

Files	Windows	Linux and macOS
Interpreter	<i>Scripts</i>	<i>bin</i>
Entry-point scripts	<i>Scripts</i>	<i>bin</i>
Third-party packages	<i>Lib site-packages</i>	<i>lib/python3.x/site-packages^a</i>
Environment configuration	<i>pyvenv.cfg</i>	<i>pyvenv.cfg</i>

^a Fedora places third-party extension modules under *lib64* instead of *lib*.

Virtual environments have their own `python` command, which is located next to the entry-point scripts. On Linux and macOS, the command is a symbolic link to the interpreter you used to create the environment. On Windows, it’s a small wrapper executable that launches the parent interpreter.⁵

Installing packages

Virtual environments include pip as a means to install packages into them.⁶ Let's create a virtual environment, install `httpx` (an HTTP client library), and launch an interactive session. On Windows, enter the following commands:

```
> py -m venv .venv
> .venv\Scripts\python -m pip install httpx
> .venv\Scripts\python
```

On Linux and macOS, enter the following commands. There's no need to spell out the path to the interpreter if the environment uses the well-known name `.venv`. The Python Launcher for Unix selects its interpreter by default:

```
$ py -m venv .venv
$ py -m pip install httpx
$ py
```

In the interactive session, use `httpx.get` to perform a `GET` request to a web host:

```
>>> import httpx
>>> httpx.get("https://example.com/")
<Response [200 OK]>
```

Virtual environments come with the version of pip that was current when Python was released. This can be a problem when you're working with an old Python release. Create the environment with the option `--upgrade-deps` to ensure you get the latest pip release from the Python Package Index.

You can also create a virtual environment without pip using the option `-without-pip` and install packages with an external installer. If you have pip installed globally, you can pass the target environment using its `--python` option, like this:


```
$ pip --python=.venv install httpx
```

It's easy to install a package into the Python installation or per-user environment by accident—especially if you're used to invoking `pip` directly. If your Python installation isn't marked as externally managed, you may not even notice. Fortunately, you can configure `pip` to always require a virtual environment when installing packages:

```
$ pip config set global.require-virtualenv true
```

Activation scripts

Virtual environments come with *activation scripts* in the *bin* or *Scripts* directory—these scripts make it more convenient to use a virtual environment from the command line, and they're provided for a number of supported shells and command interpreters. Here's the Windows example again, this time using the activation script:

```
> py -m venv .venv
> .venv\Scripts\activate
(.venv) > py -m pip install httpx
(.venv) > py
```

Activation scripts bring three features to your shell session:

- They prepend the script directory to the `PATH` variable. This allows you to invoke `python`, `pip`, and entry-point scripts without prefixing them with the path to the environment.
- They set the `VIRTUAL_ENV` environment variable to the location of the virtual environment. Tools like the Python Launcher use this variable to detect that the environment is active.
- They update your shell prompt to provide a visual reference for which environment is active, if any. By default, the prompt uses the name of the directory where the environment is located.

TIP

You can provide a custom prompt using the option `--prompt` when creating the environment. The special value `.` designates the current directory; it's particularly useful when you're inside a project repository.

On macOS and Linux, you need to *source* the activation script to allow it to affect your current shell session. Here's an example for Bash and similar shells:

```
$ source .venv/bin/activate
```

Environments come with activation scripts for some other shells as well. For example, if you use the fish shell, source the supplied *activate.fish* script instead.

On Windows, you can invoke the activation script directly. There's an *Activate.ps1* script for PowerShell and an *activate.bat* script for *cmd.exe*. You don't need to provide the file extension; each shell selects the script appropriate for it:

```
> .venv\Scripts\activate
```

PowerShell on Windows doesn't allow you to execute scripts by default, but you can change the execution policy to something more suited to development: the `RemoteSigned` policy allows scripts written on the local machine or signed by a trusted publisher. On Windows servers, this policy is already the default. You only need to do this once—the setting is stored in the registry:

```
> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Activation scripts provide you with a `deactivate` command to revert the changes to your shell environment. It's usually implemented as a shell function and works the same on Windows, macOS, and Linux:

A look under the hood

How does Python know to import a third-party package like `httpx` from the virtual environment instead of the Python installation? The location can't be hardcoded in the interpreter binary, given that virtual environments share the interpreter with the Python installation. Instead, Python looks at the location of the `python` command you used to launch the interpreter. If its parent directory contains a `pyvenv.cfg` file, Python treats that file as a *landmark* for a virtual environment and imports third-party modules from the *site-packages* directory beneath.

This explains how you import third-party modules from the virtual environment, but how does Python find modules from the standard library? After all, they're neither copied nor linked into the virtual environment. Again, the answer lies in the `pyvenv.cfg` file: when you create a virtual environment, the interpreter records its own location under the `home` key in this file. If it later finds itself in a virtual environment, it looks for the standard library relative to that `home` directory.

NOTE

The name `pyvenv.cfg` is a remnant of the `pyvenv` script that used to ship with Python. The `py -m venv` form makes it clearer which interpreter you use to create the virtual environment—and thus which interpreter the environment itself will use.

While the virtual environment has access to the standard library in the system-wide environment, it's isolated from its third-party modules. (Although not recommended, you can give the environment access to those modules as well, using the `--system-site-packages` option when creating the environment.)

How does pip know where to install packages? The short answer is that pip asks the interpreter it's running on, and the interpreter derives the lo-

cation from its own path—just like when you import a module.⁷ This is why it's best to run pip with an explicit interpreter using the `py -m pip` idiom. If you invoke `pip` directly, the system searches your `PATH` and may come up with the entry-point script from a different environment.

Installing Applications with `pipx`

In [“Virtual Environments”](#), you saw why it makes good sense to install your projects in separate virtual environments: unlike system-wide and per-user environments, virtual environments isolate your projects, avoiding dependency conflicts.

The same reasoning applies when you install third-party Python applications—say, a code formatter like Black or a packaging manager like Hatch. Applications tend to depend on more packages than libraries, and they can be quite picky about the versions of their dependencies.

Unfortunately, managing and activating a separate virtual environment for every application is cumbersome and confusing—and it limits you to using only a single application at a time. Wouldn't it be great if we could confine applications to virtual environments and still have them available globally?

That's precisely what `pipx` does, and it leverages a simple idea to make it possible: it copies or symlinks the entry-point script for the application from its virtual environment into a directory on your search path. Entry-point scripts contain the full path to the environment's interpreter, so you can copy them anywhere you want, and they'll still work.

`pipx` in a Nutshell

Let me show you how this works in a nutshell—the following commands are for Linux or macOS. First, you create a shared directory for the entry-point scripts of your applications and add it to your `PATH` environment variable:

```
$ mkdir -p ~/.local/bin  
$ export PATH="$HOME/.local/bin:$PATH"
```

Next, you install an application in a dedicated virtual environment—I’ve chosen the Black code formatter as an example:

```
$ py -m venv black  
$ black/bin/python -m pip install black
```

Finally, you copy the entry-point script into the directory you created in the first step—that would be a script named `black` in the *bin* directory of the environment:

```
$ cp black/bin/black ~/.local/bin
```

Now you can invoke `black` even though the virtual environment is not active:

```
$ black --version  
black, 24.2.0 (compiled: yes)  
Python (CPython) 3.12.2
```

On top of this simple idea, the `pipx` project has built a cross-platform package manager for Python applications with a great developer experience.

TIP

If there’s a single Python application that you should install on a development machine, `pipx` is probably it. It lets you install, run, and manage all the other Python applications in a way that’s convenient and avoids trouble.

Installing pipx

If your system package manager distributes pipx as a package, I recommend using that as the preferred installation method, as it's more likely to provide good integration out of the box:

```
$ apt install pipx  
$ brew install pipx  
$ dnf install pipx
```

As a post-installation step, update your `PATH` environment variable to include the shared script directory, using the `ensurepath` subcommand. (If you've modified your `PATH` variable when running the commands above, open a new terminal first.)

```
$ pipx ensurepath
```

On Windows, and if your system package manager doesn't distribute pipx, I recommend installing pipx into the per-user environment, like this:

```
$ py -m pip install --user pipx  
$ py -m pipx ensurepath
```

The second step also puts the `pipx` command itself on your search path.

If you don't already have shell completion for pipx, activate it by following the instructions for your shell, which you can print with this command:

```
$ pipx completions
```

Managing Applications with pipx

With pipx installed on your system, you can use it to install and manage applications from the Python Package Index (PyPI). For example, here's how you would install Black with pipx:

```
$ pipx install black
```

You can also use pipx to upgrade an application to a new release, reinstall it, or uninstall it from your system:

```
$ pipx upgrade black  
$ pipx reinstall black  
$ pipx uninstall black
```

As a package manager, pipx keeps track of the applications it installs and lets you perform bulk operations across all of them. This is particularly useful to keep your development tools updated to the latest version and to reinstall them on a new version of Python:

```
$ pipx upgrade-all  
$ pipx reinstall-all  
$ pipx uninstall-all
```

You can also list the applications you've installed previously:

```
$ pipx list
```

Some applications support plugins that extend their functionality. These plugins must be installed in the same environment as the application. For example, the packaging managers Hatch and Poetry both come with plugin systems. Here's how you would install Hatch with a plugin that determines the package version from the version control system (see [“Single-Sourcing the Project Version”](#)):

```
$ pipx install hatch
$ pipx inject hatch hatch-vcs
```

Running Applications with pipx

The previous commands provide all the primitives to manage global developer tools efficiently, but it gets better. Most of the time, you just want to use recent versions of your developer tools. You don't want the responsibility of keeping the tools updated, reinstalling them on new Python versions, or removing them when you no longer need them. Pipx allows you to run an application directly from PyPI without an explicit installation step. Let's use the classic Cowsay app to try it:

```
$ pipx run cowsay moo
```

```
  ____
 | moo |
  ====
   \   \
    ^   ^
   (oo)\_____
  (__) \       )\\/
     ||----w |
     ||     ||
```

Behind the scenes, pipx installs Cowsay in a temporary virtual environment and runs it with the arguments you've provided. It keeps the environment around for a while,⁸ so you don't end up reinstalling applications on every run. Use the `--no-cache` option to force pipx to create a new environment and reinstall the latest version.

You may have noticed an implicit assumption in the `run` command: that the PyPI package must have the same name as the command it provides. This may seem a reasonable expectation—but what if a Python package provides multiple commands? For example, the `pip-tools` package (see [“Compiling Requirements with pip-tools and uv”](#)) provides commands named `pip-compile` and `pip-sync`.

If you find yourself in this situation, provide the PyPI name using the `--spec` option, like this:

```
$ pipx run --spec pip-tools pip-sync
```

TIP

Use `pipx run <app>` as the default method to install and run developer tools from PyPI. Use `pipx install <app>` if you need more control over application environments; for example, if you need to install plugins. (Replace `<app>` with the name of the app.)

Configuring pipx

By default, pipx installs applications on the same Python version that it runs on itself. This may not be the latest stable version, particularly if you installed pipx using a system package manager like APT. I recommend setting the environment variable `PIPX_DEFAULT_PYTHON` to the latest stable Python if that's the case. Many developer tools you run with pipx create their own virtual environments; for example, virtualenv, Nox, tox, Poetry, and Hatch all do. It's worthwhile to ensure that all downstream environments use a recent Python version by default:

```
$ export PIPX_DEFAULT_PYTHON=python3.12 # Linux and macOS
> setx PIPX_DEFAULT_PYTHON python3.12   # Windows
```

Under the hood, pipx uses pip as a package installer. This means that any configuration you have for pip also carries over to pipx. A common use case is installing Python packages from a private index instead of PyPI, such as a company-wide package repository.

You can use `pip config` to set the URL of your preferred package index persistently:

```
$ pip config set global.index-url https://example.com
```

Alternatively, you can set the package index for the current shell session only. Most pip options are also available as environment variables:

```
$ export PIP_INDEX_URL=https://example.com
```

Both methods cause pipx to install applications from the specified index.

Managing Environments with uv

The tool uv is a drop-in replacement for core Python packaging tools, written in the Rust programming language. It offers order-of-magnitude performance improvements over the Python tools it replaces, in a single static binary without dependencies. While its `uv venv` and `uv pip` sub-commands aim for compatibility with virtualenv and pip, uv also embraces evolving best practices, such as operating in a virtual environment by default.

Install uv with pipx:

```
$ pipx install uv
```

By default, uv creates a virtual environment using the well-known name `.venv` (you can pass another location as an argument):

```
$ uv venv
```

Specify the interpreter for the virtual environment using the `--python` option with a specification like `3.12` or `python3.12`; a full path to an interpreter also works. Uv discovers available interpreters by scanning your `PATH`. On Windows, it also inspects the output of `py --list-paths`. If you don't specify an interpreter, uv defaults to `python3` on Linux and macOS, and `python.exe` on Windows.

NOTE

Despite its name, `uv venv` emulates the Python tool `virtualenv`, not the built-in `venv` module. `Virtualenv` creates environments with any Python interpreter on your system. It combines interpreter discovery with aggressive caching to make this fast and flawless.

By default, `uv` installs packages into the environment named `.venv` in the current directory or one of its parent directories (using the same logic as the Python Launcher for Unix):

```
$ uv pip install httpx
```

You can install packages into another environment by activating it—this works for both virtual environments (`VIRTUAL_ENV`) and Conda environments (`CONDA_PREFIX`). If there's neither an active environment nor a `.venv` directory, `uv` bails out with an error. It will never install or uninstall packages from your global environment unless you explicitly ask it to do so using the `--system` option.

While `uv`'s initial development has focused on providing drop-in replacements for standard Python tooling, its ultimate goal is to grow into that one unified packaging tool that has eluded Python for so long—with the kind of developer experience that Rust developers love about Cargo. Even at this early stage, `uv` gives you a unified and streamlined workflow, thanks to a cohesive feature set with good defaults. And it's blazingly fast.

Finding Python Modules

Python environments consist, first and foremost, of a Python interpreter and Python modules. Consequently, there are two mechanisms that play a key role in linking a Python program to an environment: interpreter discovery and module import.

Interpreter discovery is the process of locating the Python interpreter to execute a program. You've already seen the most important methods for

locating interpreters:

- Entry-point scripts reference the interpreter in their environment directly, using a shebang or a wrapper executable (see [“Entry-point scripts”](#)).
- Shells locate the interpreter by searching directories on `PATH` for commands like `python`, `python3`, or `python3.x` (see [“Locating Python Interpreters”](#)).
- The Python Launcher locates interpreters using the Windows Registry, `PATH` (on Linux and macOS), and the `VIRTUAL_ENV` variable (see [“The Python Launcher for Windows”](#) and [“The Python Launcher for Unix”](#)).
- When you activate a virtual environment, the activation script puts its interpreter and entry-point scripts on `PATH`. It also sets the `VIRTUAL_ENV` variable for the Python Launcher and other tools (see [“Virtual Environments”](#)).

This section takes a deep dive into the other mechanism that links programs to an environment: *module import*, which is the process of locating and loading Python modules for a program.

TIP

In a nutshell, just like the shell searches `PATH` for executables, Python searches `sys.path` for modules. This variable holds a list of locations from where Python can load modules—most commonly, directories on the local filesystem.

The machinery behind the `import` statement lives in `importlib` from the standard library (see [“Inspecting Modules and Packages with `importlib`”](#)). The interpreter translates every use of the `import` statement into an invocation of the `__import__` function from `importlib`. The `importlib` module also exposes an `import_module` function that allows you to import modules whose names are only known at runtime.

Having the import system in the standard library lets you inspect and customize the import mechanism from within Python. For example, the import system supports loading modules from directories and from zip archives out of the box. But entries on `sys.path` can be anything really—

say, a URL or a database query—as long as you register a function in `sys.path_hooks` that knows how to find and load modules from these path entries.

Module Objects

When you import a module, the import system returns a *module object*, an object of type `types.ModuleType`. Any global variable defined by the imported module becomes an attribute of the module object. This allows you to access the module variable in dotted notation (`module.var`) from the importing code.

Under the hood, module variables are stored in a dictionary in the `__dict__` attribute of the module object. (This is the standard mechanism used to store attributes of any Python object.) When the import system loads a module, it creates a module object and executes the module's code using `__dict__` as the global namespace. To simplify somewhat, it invokes the built-in `exec` function like this:

```
exec(code, module.__dict__)
```

Additionally, module objects have some special attributes. For instance, the `__name__` attribute holds the fully qualified name of the module, like `email.message`. The `__spec__` module holds the *module spec*, which I'll talk about shortly. Packages also have a `__path__` attribute, which contains locations to search for submodules.

NOTE

Most commonly, the `__path__` attribute of a package contains a single entry: the directory holding its `__init__.py` file. Namespace packages, on the other hand, can be distributed across multiple directories.

The Module Cache

When you first import a module, the import system stores the module object in the `sys.modules` dictionary, using its fully qualified name as a key. Subsequent imports return the module object directly from `sys.modules`. This mechanism brings a number of benefits:

Performance

Imports are expensive because the import system loads most modules from disk. Importing a module also involves executing its code, which can further increase startup time. The `sys.modules` dictionary functions as a cache to speed things up.

Idempotency

Importing modules can have side effects; for example, when executing module-level statements. Caching modules in `sys.modules` ensures that these side effects happen only once. The import system also uses locks to ensure that multiple threads can safely import the same module.

Recursion

Modules can end up importing themselves recursively. A common case is *circular imports*, where module `a` imports module `b`, and `b` imports `a`. The import system supports this by adding modules to `sys.modules` *before* they're executed. When `b` imports `a`, the import system returns the (partially initialized) module `a` from the `sys.modules` dictionary, thereby preventing an infinite loop.

Module Specs

Conceptually, Python imports a module in two steps: *finding* and *loading*. First, given the fully qualified name of a module, the import system locates the module and produces a module spec (`importlib.machinery.ModuleSpec`). Second, the import system creates a module object from the module spec and executes the module's code.

The module spec is the link between those two steps. A *module spec* contains metadata about a module such as its name and location, as well as an appropriate loader for the module (Table 2-5). You can also access most of the metadata from the module spec using special attributes directly on the module object.

Table 2-5. Attributes of module specs and module objects

Spec attribute	Module attribute	Description
name	__name__	The fully qualified name of the module
loader	__loader__	A loader object that knows how to execute the module’s code
origin	__file__	The location of the module
submodule_search_locations	__path__	Where to search for submodules, if the module is a package
cached	__cached__	The location of the compiled byte-code for the module
parent	__package__	The fully qualified name of the containing package

The `__file__` attribute of a module typically holds the filename of the Python module. In special cases, it’s a fixed string, like `"builtin"` for built-in modules, or `None` for namespace packages (which don’t have a single location).

Finders and Loaders

The import system finds and loads modules using two kinds of objects. *Finders* (`importlib.abc.MetaPathFinder`) are responsible for locating modules given their fully qualified names. When successful, their `find_spec` method returns a module spec with a loader; otherwise, it returns `None`. *Loaders* (`importlib.abc.Loader`) are objects with an `exec_module` function that load and execute the module’s code. The func-

tion takes a module object and uses it as a namespace when executing the module. The finder and loader can be the same object, which is then known as an *importer*.

Finders are registered in the `sys.meta_path` variable, and the import system tries each finder in turn. When a finder has returned a module spec with a loader, the import system creates and initializes a module object, then passes it to the loader for execution.

By default, the `sys.meta_path` variable contains three finders, which handle different kinds of modules (see [“Python modules”](#)):

- `importlib.machinery.BuiltinImporter` for built-in modules
- `importlib.machinery.FrozenImporter` for frozen modules
- `importlib.machinery.PathFinder` to search modules on `sys.path`

The `PathFinder` is the central hub of the import machinery. It’s responsible for every module that’s not embedded into the interpreter, and searches `sys.path` to locate it.⁹ The path finder uses a second level of finder objects known as *path entry finders* (`importlib.abc.PathEntryFinder`), each of which finds modules under a specific location on `sys.path`. The standard library provides two types of path entry finders, registered under `sys.path_hooks`:

- `zipimport.zipimporter` to import modules from zip archives
- `importlib.machinery.FileFinder` to import modules from a directory

Typically, modules are stored in directories on the filesystem, so `PathFinder` delegates its work to a `FileFinder`. The latter scans the directory for the module and uses its file extension to determine the appropriate loader. There are three loaders for the different kinds of modules:

- `importlib.machinery.SourceFileLoader` for pure Python modules
- `importlib.machinery.SourcelessFileLoader` for bytecode modules
- `importlib.machinery.ExtensionFileLoader` for binary extension modules

The zip importer works similarly, except that it doesn't support extension modules because current operating systems don't allow loading dynamic libraries from a zip archive.

The Module Path

When your program can't find a specific module or imports the wrong version of a module, it can help to look at `sys.path`, the module path. But where do the entries on `sys.path` come from in the first place? Let's unravel some of the mysteries of the module path.

When the interpreter starts up, it constructs the module path in two steps. First, it builds an initial module path using some built-in logic. Most importantly, this initial path includes the standard library. Second, the interpreter imports the `site` module from the standard library. The `site` module extends the module path to include the site packages from the current environment.

In this section, we'll take a look at how the interpreter constructs the initial module path with the standard library. The next section explains how the `site` module appends directories with site packages.

NOTE

You can find the built-in logic for constructing `sys.path` in *Modules/getpath.py* in the CPython source code. Despite appearances, this is not an ordinary module. When you build Python, its code is frozen to bytecode and embedded in the executable.

The locations on the initial module path fall into three categories, and they occur in this order:

1. The current directory or the directory containing the Python script (if any)
2. The locations in the `PYTHONPATH` environment variable (if set)
3. The locations of the standard library

Let's look at each in more detail.

The current directory or the directory containing the script

The first item on `sys.path` can be any of the following:

- If you ran `py <script>`, the directory where the script is located
- If you ran `py -m <module>`, the current directory
- Otherwise, the empty string, which also denotes the current directory

Traditionally, this mechanism has long provided a convenient way to structure an application: just put the main entry-point script and all application modules in the same directory. During development, launch the interpreter from within that directory for interactive debugging, and your imports will still work.

Unfortunately, having the working directory on `sys.path` is quite unsafe, as an attacker (or you, mistakenly) can override the standard library by placing Python files in the victim's directory. To avoid this, starting with Python 3.11, you can use the `-P` interpreter option or the `PYTHONSAFEPATH` environment variable to omit the current directory from `sys.path`. If you invoke the interpreter with a script, this option also omits the directory where the script is located.

Installing your application into a virtual environment is a safer and more flexible approach than putting its modules in the current directory. This requires packaging the application, which is the topic of [Chapter 3](#).

The PYTHONPATH variable

The `PYTHONPATH` environment variable provides another way to add locations before the standard library on `sys.path`. It uses the same syntax as the `PATH` variable. Avoid this mechanism for the same reasons as the current working directory and use a virtual environment instead.

The standard library

[Table 2-6](#) shows the remaining entries on the initial module path, which are dedicated to the standard library. Locations are prefixed with the path to the installation, and may differ in details on some platforms. Notably, Fedora places the standard library under *lib64* instead of *lib*.

Table 2-6. The standard library on `sys.path`

Windows	Linux and macOS	Description
<i>python3x.zip</i>	<i>lib/python3x.zip</i>	For compactness, the standard library can be installed as a zip archive. This entry is present even if the archive doesn't exist (which it normally doesn't).
<i>Lib</i>	<i>lib/python3.x</i>	Pure Python modules
<i>DLLs</i>	<i>lib/python3.x/lib-dynload</i>	Binary extension modules

The location of the standard library is not hardcoded in the interpreter (see [“Virtual Environments”](#)). Rather, Python looks for landmark files on the path to its own executable and uses them to locate the current environment (`sys.prefix`) and the Python installation (`sys.base_prefix`). One such landmark file is *pyvenv.cfg*, which marks a virtual environment and points to its parent installation via the `home` key. Another landmark is *os.py*, the file containing the standard `os` module: Python uses *os.py* to discover the prefix outside a virtual environment and to locate the standard library itself.

Site Packages

The interpreter constructs the initial `sys.path` early on during initialization using a fairly fixed process. By contrast, the remaining locations on `sys.path`—known as *site packages*—are highly customizable and under the responsibility of a Python module named `site`.

The `site` module adds the following path entries if they exist on the filesystem:

User site packages

This directory holds third-party packages from the per-user environment. It's in a fixed location that depends on the OS (see [“The Per-User Environment”](#)). On Fedora and some other systems, there are two path entries: for pure Python modules and extension modules, respectively.

Site packages

This directory holds third-party packages from the current environment, which is either a virtual environment or a system-wide installation. On Fedora and some other systems, pure Python modules and extension modules are in separate directories. Many Linux systems also separate distribution-owned site packages under `/usr` from local site packages under `/usr/local`.

In the general case, the site packages are in a subdirectory of the standard library named `site-packages`. If the `site` module finds a `pyvenv.cfg` file on the interpreter path, it uses the same relative path as in a system installation but starts from the virtual environment marked by that file. The `site` module also modifies `sys.prefix` to point to the virtual environment.

The `site` module provides a few hooks for customization:

.pth files

Within site packages directories, any file with a `.pth` extension can list additional directories for `sys.path`, one directory per line. This works similarly to `PYTHONPATH`, except that modules in these directories will never shadow the standard library. Additionally, `.pth` files can import modules directly—the `site` module executes any line starting with `import` as Python code. Third-party packages can ship `.pth` files to configure `sys.path` in an environment. Some packaging tools use `.pth` files behind the scenes to implement editable installs. An *editable install* places the source directory of your

project on `sys.path`, making code changes instantly visible inside the environment.

The `sitecustomize` module

After setting up `sys.path` as described previously, the `site` module attempts to import the `sitecustomize` module, typically located in the *site-packages* directory. This provides a hook for the system administrator to run site-specific customizations when the interpreter starts up.

The `usercustomize` module

If there is a per-user environment, the `site` module also attempts to import the `usercustomize` module, typically located in the user *site-packages* directory. You can use this module to run user-specific customizations when the interpreter starts up. Contrast this with the `PYTHONSTARTUP` environment variable, which allows you to specify a Python script to run before interactive sessions, within the same namespace as the session.

If you run the `site` module as a command, it prints out your current module path as well as some information about the per-user environment:

```
$ py -m site
sys.path = [
    '/home/user',
    '/usr/local/lib/python312.zip',
    '/usr/local/lib/python3.12',
    '/usr/local/lib/python3.12/lib-dynload',
    '/home/user/.local/lib/python3.12/site-packages',
    '/usr/local/lib/python3.12/site-packages',
]
USER_BASE: '/home/user/.local' (exists)
USER_SITE: '/home/user/.local/lib/python3.12/site-packages' (exists)
ENABLE_USER_SITE: True
```

Back to the Basics

If you've read this far, the module path may seem a little—*byzantine*?

Here's a good, solid intuition of how Python locates a module: the interpreter searches the directories on `sys.path` for the module—first the directories that contain the modules of the standard library, then the *site-packages* directory with third-party packages. An interpreter in a virtual environment uses the *site-packages* directory from that environment.

As you've seen in this section, the truth is far more complex than that simple story. But I've got good news for you: Python lets you *make* that story true. The `-P` interpreter option omits the directory containing your script from the module path (or the current directory, if you're running your program with `py -m <module>`). The `-I` interpreter option omits the per-user environment from the module path as well as any directories set with `PYTHONPATH`. Use both options when running your Python programs if you want a more predictable module path.

If you re-run the `site` module with the `-I` and `-P` options, the module path is cut down to just the standard library and site packages:

```
$ py -IPm site
sys.path = [
    '/usr/local/lib/python312.zip',
    '/usr/local/lib/python3.12',
    '/usr/local/lib/python3.12/lib-dynload',
    '/usr/local/lib/python3.12/site-packages',
]
USER_BASE: '/home/user/.local' (exists)
USER_SITE: '/home/user/.local/lib/python3.12/site-packages' (exists)
ENABLE_USER_SITE: False
```

The current directory no longer appears on the module path, and the per-user site packages are gone, too—even though the directory exists on this system.

Summary

In this chapter, you’ve learned what Python environments are, where to find them, and how they look on the inside. At the core, a Python environment consists of the Python interpreter and Python modules, as well as entry-point scripts to run Python applications. Environments are tied to a specific version of the Python language.

There are three kinds of Python environments. *Python installations* are complete, stand-alone environments with an interpreter and the full standard library. *Per-user environments* are annexes to an installation where you can install modules and scripts for a single user. *Virtual environments* are lightweight environments for project-specific modules and entry-point scripts, which reference their parent environment via a *pyvenv.cfg* file. They come with an interpreter, which is typically a symbolic link or small wrapper for the parent interpreter, and with activation scripts for shell integration. You create a virtual environment using the command `py -m venv .`

Install Python applications with `pipx` to make them available globally while keeping them in separate virtual environments. You can install and run an application using a single command, such as `pipx run black`. Set the `PIPX_DEFAULT_PYTHON` variable to ensure `pipx` installs tools on the current Python release.

`Uv` is a blazingly fast drop-in replacement for `virtualenv` and `pip` with better defaults. Use `uv venv` to create a virtual environment, and `uv pip` to install packages into it. Both commands use the `.venv` directory by default, just like the `py` tool on Unix. The `--python` option lets you select the Python version for the environment.

In the final section of this chapter, you’ve learned how Python uses `sys.path` to locate modules when you import them and how the module path is constructed during interpreter startup. You’ve also learned how module import works under the hood, using finders and loaders as well as the module cache. Interpreter discovery and module import are the

key mechanisms that link Python programs to an environment at runtime.

- 1 There's also a `pythonw.exe` executable that runs programs without a console window, like GUI applications.
- 2 A *shared library* is a file with executable code that multiple programs can use at runtime. The operating system keeps only a single copy of the code in memory.
- 3 Windows installations don't include an entry-point script for `pydoc`—launch it using `py -m pydoc` instead.
- 4 Historically, macOS framework builds pioneered per-user installation before it became a standard in 2008.
- 5 You could force the use of symbolic links on Windows via the `--symlinks` option—but don't. There are subtle differences in the way these work on Windows. For example, the File Explorer resolves the symbolic link before it launches Python, which prevents the interpreter from detecting the virtual environment.
- 6 Before Python 3.12, the `venv` module also pre-installed `setuptools` for the benefit of legacy packages that don't declare it as a build dependency.
- 7 Internally, pip queries the `sysconfig` module for an appropriate *installation scheme*—a Python environment layout. This module constructs the installation scheme using the build configuration of Python and the location of the interpreter in the filesystem.
- 8 At the time of writing in 2024, pipx caches temporary environments for 14 days.
- 9 For modules located within a package, the `__path__` attribute of the package takes the place of `sys.path`.